

Where the Time Goes: CPU and GPU Rendering in a Narrated-Video Pipeline

Rajesh Kumar Kona

Voice-to-Video, an independent open-source project · <https://github.com/0KRK0/Voice-to-Video>

Abstract

Voice-to-Video turns a spoken or written explanation into a narrated video. Its renderer composes every frame in Python with Pillow and encodes with x264 through FFmpeg. Rendering is the only cost in the product that grows with the length of a video, so this report asks where acceleration effort should go. A profile of a 1080p render put 77% of the time in frame composition and 23% in encoding, and inside composition only 0.7% in text. By Amdahl's law, a hardware encoder can speed up that profile by at most $1.3\times$. We therefore (i) removed redundant full-frame buffer work, which took composition from 12 to 26 frames per second on unchanged hardware; (ii) split the render into checkpointed, independently encoded segments of about 12 seconds, which makes the render both resumable and parallel across processes; and (iii) built an OpenGL 3.3 compositor that moves only photograph resampling to the graphics card. It uses a two-pass Lanczos-3 shader with an 8-bit intermediate so that it reproduces Pillow's output. Correctness is defined as a hard per-pixel bound: no channel may differ from the CPU reference by more than 2, and no pixel may exceed that. On a GeForce GTX 1650, the first hardware run passed 15 of 23 calibration scenes; the pixel counts pointed to four defects. After fixing them, all 23 scenes pass on a detailed photograph, with a worst difference of 2 and a mean absolute difference of at most 0.07. Two- and four-hour renders completed with flat memory (slopes of -0.095 and -0.113 MB per segment) and zero duration error. We did not take a controlled CPU-versus-GPU throughput measurement, and this report says so rather than estimating one.

1 Introduction

A narrated explainer is a long sequence of short shots: photographs moving under a slow camera, drawn charts and diagrams, typography, captions and credit chips, joined by dissolves, wipes and pushes, all timed to a single narration track. Voice-to-Video (VTV) produces such videos from a recording, a script or a document. Every upstream stage is either fixed per project (transcription, planning, sourcing) or cheap. Rendering is not: its cost is proportional to the number of frames, and a forty-minute video at 30 frames per second is 72,000 of them.

The obvious product move is a “Fast (GPU)” switch that hands encoding to NVENC or a similar hardware encoder. This report records why the project did not do that. It also records what was done instead, and how the result was checked. Its contributions are:

1. a measured profile of a Python/Pillow compositor feeding x264, and an Amdahl analysis showing that hardware encoding cannot pay for itself on that profile (§4, §5);
2. a segmented rendering design in which checkpointing, resumption and parallelism come from one decomposition, with frame-exact partitioning so the video cannot drift from its audio (§6);
3. a narrow GPU compositor that reproduces Pillow's Lanczos resampling to within two levels per channel, with the reasons each design choice was forced (§7);
4. an equivalence methodology with a zero-outlier rule, and a diagnosis in which the pixel counts fully explained the failures of the first hardware run (§8, §9);
5. long-render validation at two and four hours (§9.3), and an explicit list of what has *not* been measured (§11).

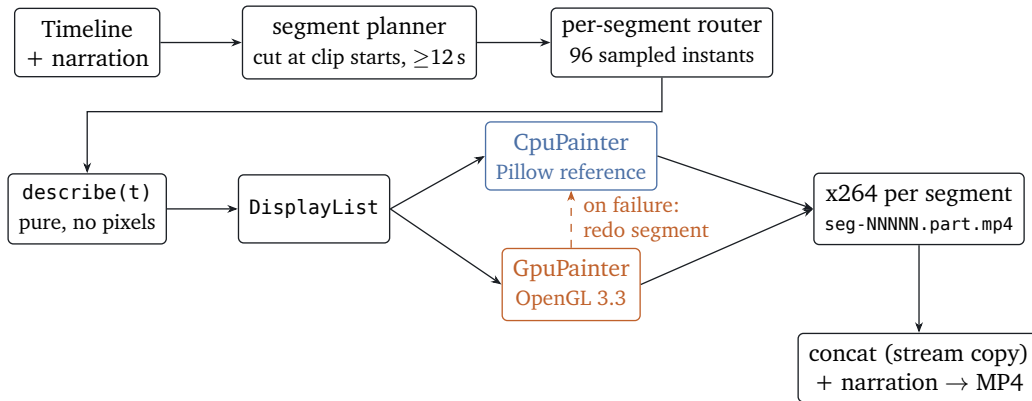


Figure 1: Render architecture. Both painters consume the same display list. A segment is routed to the CPU unless some sampled instant needs photograph resampling. A GPU failure costs one segment, which is redrawn by the reference painter.

Every number in this report is taken from an artefact in the public repository: source code, documentation, a JSON report, the diagnosis images or the project database. Appendix A maps each number to its source.

2 System overview

VTV is a Python 3.11 system of 173 source modules (about 63,500 lines) with 64 test files containing 1,708 test functions. Its layers are strict: contracts, then ports, then adapters, then services, then apps, and the dependency rule is enforced by tests. A project passes through capture, transcription, understanding, scene planning, visual direction, a grounding gate, asset resolution and composition, and ends in rendering. Rendering consumes a resolved Timeline of clips, captions and one narration file.

Two early decisions shape everything below. First, *the narration is the only clock*: visuals are fitted to the voice, never the reverse. Second, the renderer *draws every frame itself* instead of stitching encoded clips. Transitions become a blend of two frames, captions share one type system with everything else, and there is no second timeline for audio to drift against.

Describe, then paint. The composer is split into a pure function and a painter (Figure 1). `describe(t)` returns a `DisplayList`, an ordered list of layers (Background, Picture, Drawn, Plate, Label, Vignette, Message). It keeps overlays in a separate list so that a transition never mixes captions or credits. It also carries the outgoing frame for a transition and the eased progress of the mix. It touches no pixels, fonts or files. A Painter turns that list into an RGB frame. The invariant is that *every painter consumes the same DisplayList*. If a backend needed its own description, the composer would have to know which backend it was describing for, and the seam would be gone.

3 The baseline: one process, one pass

The first renderer drew each frame with Pillow and piped raw pixels to a single FFmpeg process that also muxed the narration:

```
ffmpeg -f rawvideo -pix_fmt rgb24 -s WxH -r fps -i - -i narration -c:v libx264
-preset veryfast -crf {18|23|30} -af "adelay={ms}:all=1,apad" -c:a aac -shortest out.mp4
```

The design is simple and correct, and `apad` before `-shortest` keeps deliberate visual time after the voice ends. It had two flaws that matter in a product. A render was all-or-nothing: a crash at 97% of a three-hour encode started again from frame zero. And it used one core. A forty-minute video

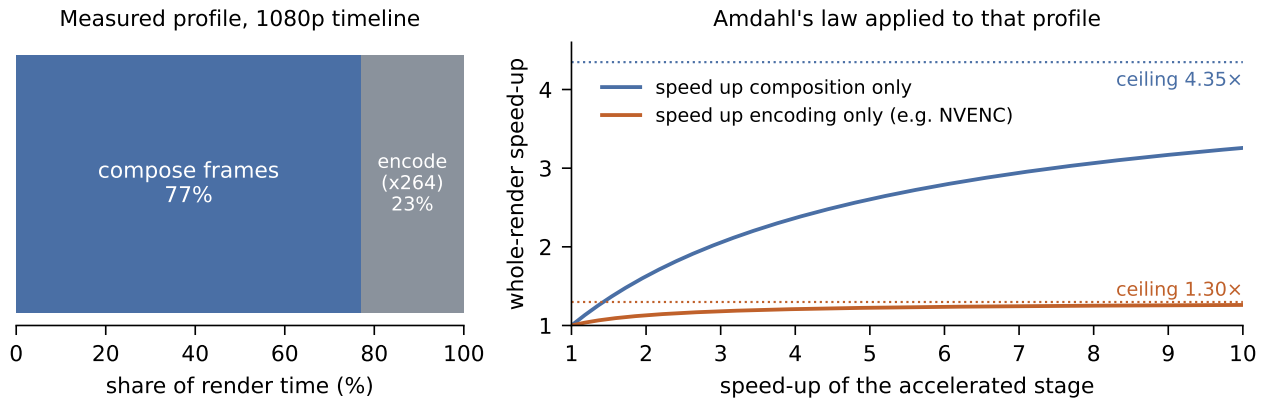


Figure 2: Left: measured share of render time. Right: whole-render speed-up predicted by Amdahl's law from that profile, when only composition or only encoding is accelerated. The curves are analytical and derived from the measured 77/23 split.

took about four times its own length to render while the other cores sat idle. Threads could not fix this, because composition is Pillow and Python, and the global interpreter lock serialises exactly the part that is slow. An August 2026 internal audit also found renders running inside the API's event loop, so that a single 25–30 s render stalled every request. Rendering was then moved to a separate worker process behind a durable SQLite queue.

4 Profiling: where the time goes

Before choosing an accelerator, a real 1080p timeline was profiled (Figure 2, left).

- **Composition was 77% of render time and encoding 23%.**
- **Inside composition, glyph rasterisation was 0.7%.** The cost was moving 1920×1080 buffers around: a paste, a blend, a channel split and a colour conversion, per frame.
- Most of that buffer work was *redundant*. Removing it took composition from **12 to 26 frames per second** on unchanged hardware. That is a $2.2\times$ gain in composition alone, more than a hardware encoder could ever give the whole render.

Two further CPU optimisations came from the same method. A *plain-frame fast path* returns frames made of a single drawn, picture or message layer directly, skipping an RGBA round trip worth about 10 ms of a 42 ms frame at 1080p. The *vignette* mask (an ellipse blurred with a Gaussian of radius $0.08w$, 154 px at 1080p) cost 19 ms per frame, which is 43 s over a 76-second video at 30 fps. It is now cached, and on opaque frames it is applied as a single alpha composite. Both fast paths are covered by tests that check they are equivalent to the slow paths.

5 Why not hardware encoding

Hardware encoders (NVENC, Quick Sync, VAAPI, VideoToolbox, AMF) replace only the 23%. By Amdahl's law [1], an encoder of infinite speed takes a render to 77% of its time, a ceiling of $1/0.77 \approx 1.3\times$ (Figure 2, right). Accelerating composition instead has a ceiling of $1/0.23 \approx 4.3\times$. The hardware path also conflicts with the parallelism in §6. Segments run in a process pool with one encoder each, and consumer NVIDIA drivers have historically capped concurrent NVENC sessions at three to five, so eight parallel segments would fail or queue on the card. The system therefore *reports* which hardware encoders the local FFmpeg can open (it probes each one by encoding a single frame), but encoding stays on the CPU until a measurement says otherwise. This decision depends on the profile. Once composition moves to the GPU, the share taken by encoding grows, and the question should be asked again (§12).

6 Segmented rendering

The rule. *A frame range that has been encoded is a file on disk whose name says so. A segment is finished if and only if `seg-NNNNN.mp4` exists. That name is only ever created by `os.replace` from a `.part.mp4` after FFmpeg exits with status zero. Resumption is not a separate recovery routine. Every render asks of every segment “is this already done?”, and a first attempt is the case where the answer is always no.*

Boundaries. Segments are cut only at clip starts, accumulating until a segment holds at least $\text{round}(12 \cdot \text{fps})$ frames. Twelve seconds bounds the work lost in a crash. It splits a forty-minute video into about two hundred pieces, enough to keep eight cores busy to the end. It keeps FFmpeg’s start-up cost of about 30 ms under half a percent of each segment. The arithmetic is done in *frame indices*, not seconds: each boundary is $\text{round}(\text{start} \cdot \text{fps})$ and each segment runs to the next boundary, so the segments partition $\text{range}(\text{total_frames})$ exactly. Rounding each segment’s own duration would gain or lose a frame per segment, and a two-hundred-segment video would drift seconds from its audio.

Invalidation. The scratch directory is named by a fingerprint: SHA-256 of the timeline and settings with only the timestamp fields removed, truncated to 20 hex digits. A retry of the same work finds its own segments, and any edit lands in a fresh directory. The fingerprint deliberately over-invalidates. Re-rendering unnecessarily costs time, but reusing a stale segment ships the wrong picture, and those risks are not symmetrical.

Parallelism and assembly. Because `describe` is a pure function of the timeline and a time, frame 100,000 can be drawn without frame 99,999. Segments are therefore also the unit of parallel work. They run in a `ProcessPoolExecutor` of $\min(8, \text{cores}, \text{outstanding})$ workers, capped at eight because past that a 1080p render waits on the encoder and the page cache. Each segment is encoded with `-an -c:v libx264 -preset veryfast -crf c -pix_fmt yuv420p -threads 1`. The final file is a concat-demuxer stream copy (`-c:v copy -movflags +faststart`), with the narration muxed in at the same step. A partial preview of the finished prefix is republished every 30 s. In a container, a 30-second render finishes in about 21 s on eight cores.

Evidence. Tests compare decoded frames with `ffmpeg -f framemd5` and check that a resumed render equals an uninterrupted one, that a pooled render equals an inline one, and that `.part` or empty files are never counted as finished. A separate harness kills the desktop renderer mid-render and fails if any finished segment’s modification time changes during the resume.

7 GPU compositing

7.1 What moves to the card, and why so little

The GPU painter uses OpenGL 3.3 through `moderngl`, in a standalone context with no window. OpenGL 3.3 is available on Windows machines from NVIDIA, AMD and Intel with drivers from the last decade. It is a single pip wheel rather than a vendor SDK, it is not CUDA-specific, and the back-end is one file that could later be swapped for `wgpu`. The division of work is deliberately lopsided (Table 1).

Each part that stayed on the CPU stayed there because of a measurement. Text is 0.7% of composition, and rasterising glyphs with the reference on both paths makes a caption bit-identical between the painters. Overlays went back to the reference after the first hardware run (§9). Pillow’s RGBA drawing *replaces* pixels and flattens onto the theme background, where a GPU alpha-composites;

Work	CPU (Pillow reference)	GPU (OpenGL 3.3)
Layout, text, plates, rounded rectangles	✓	
Animation engine (charts, diagrams, typography)	✓	
Overlays (captions, credits, badges)	✓	
Transition mix (blend, wipe, push)	✓	
Photograph resampling and placement (camera moves, crop)		✓
Encoding (x264)	✓	

Table 1: Division of work in the current GPU painter.

reproducing that quirk in a shader would mean re-implementing a behaviour nobody chose. Transitions went back because the reference wipe is a hard step with a blurred band only in between, and doing it on the card cost three readbacks and two uploads to replace one `Image.blend`. What remains is the expensive, embarrassingly parallel part: a photograph is almost always larger than the frame, so every picture is a Lanczos reduction, once per frame, per shot.

7.2 Matching Pillow’s resampler

A GPU’s built-in bilinear filter samples only the four texels nearest each output pixel, however large the reduction. On detail finer than the reduction factor this aliases, and fine stripes shimmer as a camera move drifts across them. On the calibration still, Lanczos against bilinear for a 0.8 downscale differed by up to 40 levels per channel, with a mean of 8. The shader therefore implements Lanczos-3 [2, 3],

$$L_3(x) = \begin{cases} \frac{3 \sin(\pi x) \sin(\pi x/3)}{\pi^2 x^2} & 0 < |x| < 3 \\ 1 & x = 0 \\ 0 & |x| \geq 3, \end{cases}$$

and follows Pillow’s `ImagingResampleHorizontal` in the details that change pixels. Output pixel i maps to source coordinate $(i + 0.5) \cdot s$. When reducing, the kernel is widened by $\max(1, s)$; without this a downscale is undersampled whatever the kernel. Taps outside the image are *truncated* and the surviving weights renormalised. An earlier version that clamped taps to the edge texel duplicated it and produced a different picture along every border.

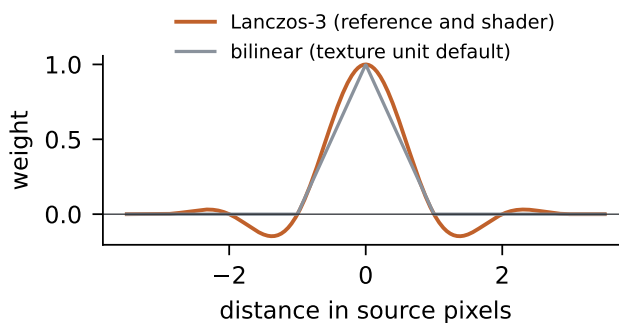


Figure 3: Lanczos-3 (used by the reference and the shader) against the bilinear tent that a texture unit uses by default.

Two passes through an 8-bit buffer. Pillow resizes in two one-dimensional passes and *writes an 8-bit image between them*. Lanczos overshoots at sharp edges, and the 8-bit clip discards that overshoot before the second pass sees it. A first shader did the whole kernel in one 2D pass in floating point. It was checked against Pillow on a gradient and on low-contrast stripes (worst difference 1) and looked correct, but neither input rings out of range. On a still with pure white lines it was off by 19

to 23, with hundreds of pixels outside tolerance. Measured on high-contrast material, one float pass is off by up to 60; two passes through a clipped RGBA8 intermediate are off by 0. The intermediate framebuffer is RGBA8 on purpose, because the clamp and the quantisation are part of the behaviour being reproduced. The remaining gap is arithmetic: Pillow rounds weights to 22-bit fixed point and accumulates in integers, while the shader uses float32. The shader loop is capped at 64 taps, so reductions beyond $10\times$ send the whole frame to the CPU painter.

7.3 Routing, capability and fallback

Routing. `DisplayList.needs_resampling` is true only if a Picture appears in the frame, or in the outgoing side of a transition. The same predicate is used by the painter and by the segment router. The router samples 96 instants across a segment plus its last frame, one every eighth of a second over twelve seconds. It puts the CPU first in the chain unless some instant needs resampling, so typography-only segments never pay for an upload.

Capability is not presence. A card can be present but driverless, too old for the shaders, inside a container with no render node, or out of memory. All of these read as “a GPU is installed” and fail on the first frame of a long render. The probe therefore initialises a context, compiles the pipelines, *draws a 320×180 calibration frame, and compares it with the CPU reference* under the rule of §8. Only a passing device is registered. The result is cached against a fingerprint of platform, vendor, renderer, driver and API, so a driver update invalidates it. `VTV_DISABLE_GPU` can only turn the GPU off; a switch that forces a capability on is how a render ends up unable to start.

Fallback. A GPU failure means “this backend cannot do this segment”, never “the project is broken”. The GPU backend marks driver resets, lost contexts, exhausted memory and shader failures as retry-elsewhere, and the segment is redrawn by the next backend in the chain. Because segments are checkpointed, a failure costs one segment. Every fallback is reported in the job’s event stream, and the job counts segments per backend. A dedicated harness renders the same video three ways: CPU-only; GPU-only with no fallback; and GPU with a device loss injected at a chosen segment. It checks with `framemd5` that the segments redrawn on the CPU are byte-identical to a CPU-only render.

8 Defining “the same picture”

GPU rasterisation is not bit-exact with Pillow: texture addressing and floating-point arithmetic are implementation-defined, and an edge pixel landing on 127 rather than 128 is not a defect. The acceptance rule therefore has two parts:

1. no channel of any pixel may differ from the reference by more than 2 (on a 0–255 scale); and
2. no pixel may exceed that at all (`MAX_OUTLIERS = 0`).

The second part is the one that catches the subtle errors. A mean-difference test would pass a caption sitting one pixel high on every frame of a four-hour video. Gross errors, such as a misplaced plate, a missing overlay or a wipe running backwards, differ by hundreds, and either part catches them. The harness renders 23 scenes covering every layer type, every transition at 0%, 50% and 100% progress, and overlays over a picture and during a blend. It reports the worst per-channel difference, the count of pixels outside tolerance, the mean absolute difference and a bounding box.

A classifier then explains failures by transforming the GPU frame until the difference collapses. It tries vertical and horizontal flips, ± 1 px shifts, a uniform offset (a wrong clear colour), edges only (texture filtering) and high frequencies only. If flipping takes the worst difference from 184 to 0, the cause is a mirrored readback, not a matter of opinion. Both the harness and the classifier were tested before being trusted. Pointed at deliberately sabotaged painters (one shifting a plate by 2 px, one dropping overlays, one raising), they must catch every fault; pointed at two reference painters,

they must pass. A NumPy transcription of the shader is also checked against Pillow’s LANCZOS at four target sizes, and tests confirm that bilinear filtering and a single float pass fail the rule.

9 Results

All GPU results below come from one machine: an NVIDIA GeForce GTX 1650, OpenGL 3.3.0 driver 610.88, Windows 10 build 26200, Python 3.11.4.

9.1 First contact: 15 of 23

The first hardware run passed 15 of the 23 scenes. The *pattern* of failures gave the diagnosis before any pixel was inspected. Every failing scene contained something vertically asymmetric (a plate near the top, a line of text, a message card), and every passing one was symmetric (a flat background, a centred vignette, a full-frame picture, a horizontal wipe). The arithmetic confirmed it: a plate at rows 11–50 of a 180-row frame differed across rows 11–169, which is the plate and its mirror and nothing between. Table 2 lists the four defects.

#	Defect	Evidence	Fix
1	Framebuffer read bottom-up (OpenGLs origin is lower-left, Pillow’s upper-left)	7 of 8 failures; bounding box exactly plate $\cup$ mirror(plate)	flip on readback
2	Cleared to black, whereas the reference starts from the theme background (11, 13, 16)	18,432 of the 20,736 differing pixels in picture-contained	clear to the theme background
3	Picture drawn at its box, but the reference scales to cover and clamps the origin, overflowing one edge	the remaining 2,304 pixels, a 9-row band	<code>_fitted_box</code> reproduces the arithmetic
4	Overlays alpha-composited, whereas the reference replaces pixels and flattens onto the theme colour	<code>overlay-*</code> worst difference 193	overlays drawn by the reference painter

Table 2: Defects found on first contact with real hardware. For picture-contained, $18,432 + 2,304 = 20,736$ equals the reported outlier count exactly, so defects 2 and 3 account for every failing pixel in that scene.

9.2 After the fixes: 23 of 23 on a detailed photograph

The diagnosis was then re-run with the detailed still (`--photo`), which adds real resampling to the geometry test. Every scene passed (Figure 4): worst per-channel difference 2, **zero pixels outside tolerance**, and mean absolute difference at most 0.07. Scenes with no photograph (background, message, plates, text, vignettes) were bit-identical. All non-zero differences occur in scenes that contain a resampled picture, which is consistent with the float32-versus-fixed-point residual predicted in §7. This also answers the policy question the project had left open: whether a GPU could match a Lanczos reference on a real photograph within two levels. With the two-pass, 8-bit-intermediate shader on this device, it could.

The diagnosis script writes images only for scenes that *fail*, so the passing run left none. The images that are in the repository (Figure 5) come from an earlier `--photo` run that did fail. Recomputed from those files, the worst differences are 17 to 23 levels, with 277 to 419 pixels outside tolerance per scene. That matches the code’s own account of the first, single-pass floating-point shader, which was “out by 19 to 23 with hundreds of pixels past tolerance” on this still. The out-of-tolerance pixels sit on the high-contrast white lines, which is where Lanczos overshoot would appear. Figure 5 therefore shows the failure that motivated the two-pass design, and shows the zero-outlier rule catching

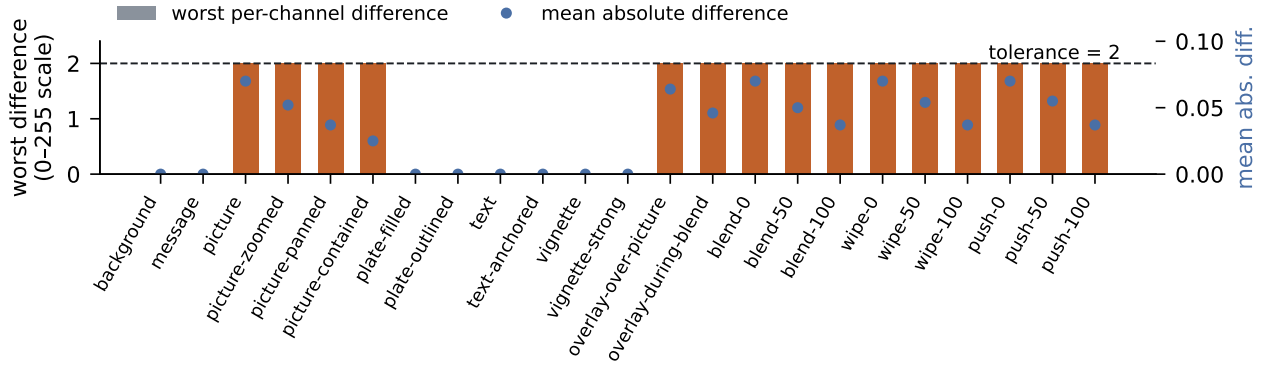


Figure 4: GPU against CPU reference, 23 calibration scenes at 320×180 , detailed photograph, GTX 1650, after the four fixes. Bars: worst per-channel difference (grey = 0). Dots: mean absolute difference. Every scene has zero pixels outside tolerance.

it.

9.3 Long renders

Before this work, the longest video rendered end to end was 150 seconds. The long-render harness renders on the desktop agent with the GPU painter available. It samples process memory, scratch disk and VRAM every two seconds, writes its report after every length, and ends each length by measuring the output with `ffprobe` against the requested duration. Its fixture alternates typography shots and a 1600×1000 test-pattern photograph, with Ken Burns, pan and zoom moves and 0.8 s dissolve, wipe and push transitions. Shots are 6 s long up to one hour and 12 s beyond. Output is 1920×1080 at 30 fps, standard quality (CRF 23).

A sixty-minute render took seventy-one minutes and drew all 296 segments correctly. It then failed to upload, because a signed URL issued when the job was claimed had expired after an hour. URLs are now issued when there is a file to send. After that fix, the two- and four-hour runs passed (Table 3, Figure 6).

Video	Render	Speed vs real time	Segments	CPU/GPU	Peak RSS	RSS slope (MB/seg.)	Peak scratch	Peak VRAM	Duration error
120 min	142.0 min	0.84×	600	2 / 598	465 MB	−0.095	4.6 GB	505 MB	0.00 s
240 min	341.4 min	0.70×	1,200	2 / 1,198	471 MB	−0.113	9.2 GB	916 MB	0.00 s

Table 3: Long-render validation on the GTX 1650 machine. Output files were 962 MB and 1,924 MB. “CPU/GPU” is the number of segments drawn by each painter.

Three findings follow. Memory is flat: peak RSS barely moves between 600 and 1,200 segments, and both slopes are slightly negative. The output is exactly as long as requested, which is the failure a progress counter cannot see. Almost every segment went to the GPU. This is consistent with the routing rule, because every typography shot in the fixture opens with a transition from a photograph, so some sampled instant in its segment needs resampling. Peak scratch disk grew roughly linearly with length (4.6 GB, then 9.2 GB), which sets a practical disk requirement for multi-hour renders.

9.4 Product renders

Figure 7 places the long renders next to the five renders recorded in the development database for real projects (1080p30, standard quality, with sourced photographs, generated images, captions and credits). These ran between $3.9 \times$ and $8.9 \times$ slower than real time; the forty-minute project (2,399 s of video) took 10,116 s, or $4.2 \times$. **This comparison is not controlled.** The product renders used real, large photographs (up to about 30 MB per file) and denser layouts, the painter and worker

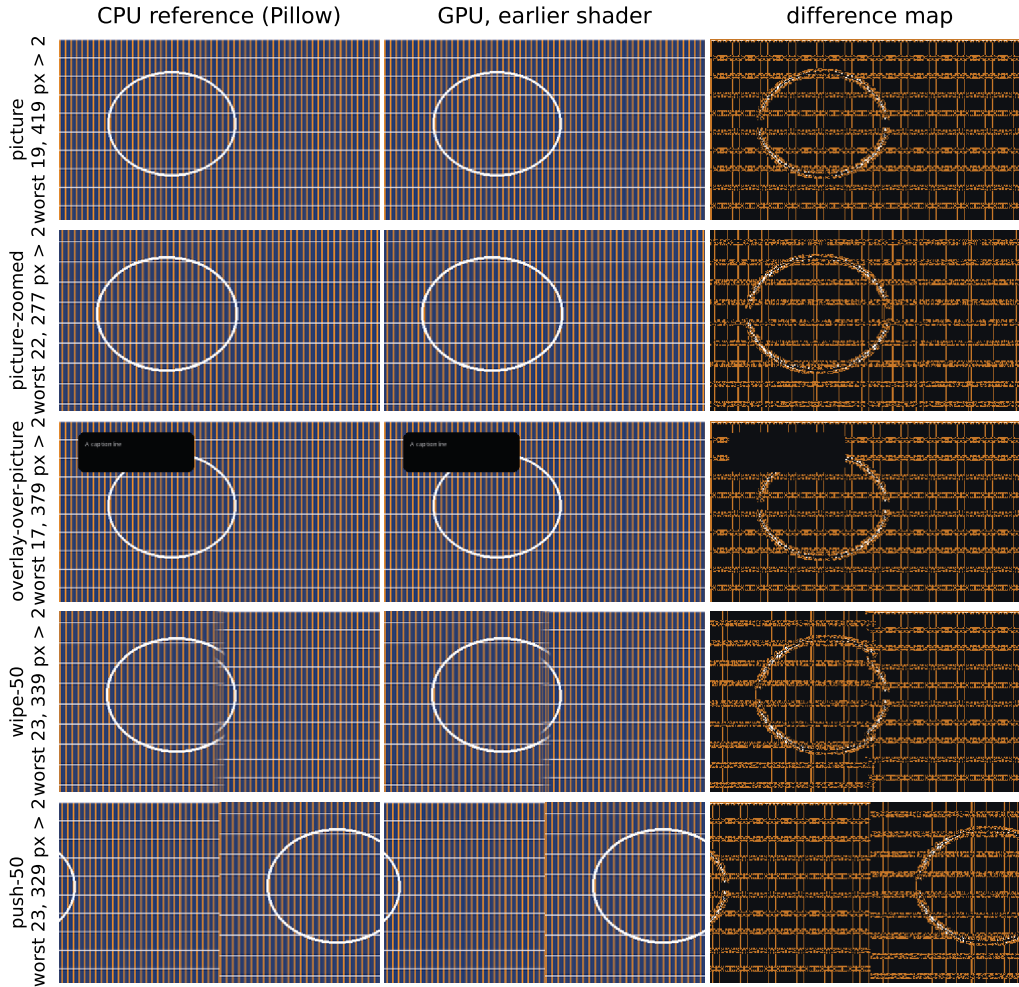


Figure 5: Frames left by an earlier, *failing* diagnosis run on the detailed still (the final, passing run writes no images). Difference map: dark means identical, amber means 1–2 levels (within tolerance), white means more than 2. The out-of-tolerance pixels lie on the high-contrast white lines, where a single floating-point pass keeps overshoot that Pillow’s 8-bit intermediate removes. Per-row figures are recomputed from the saved PNGs.

count they used were not recorded, and they predate the GPU verification. The chart shows the range of costs the system has actually paid; it does not measure a GPU speed-up.

10 Discussion: the best way to render this workload

For a Python/Pillow compositor feeding x264, the evidence supports the following order of work.

1. **Profile before accelerating.** The popular fix, hardware encoding, addresses 23% of the time and is capped at $1.3\times$. Removing redundant buffer work on the CPU gave $2.2\times$ in composition for free.
2. **Parallelise across processes, not threads, by segmenting on frame indices.** One decomposition gives resumability, parallelism and a unit of fallback, and exact frame partitioning keeps audio sync by construction.
3. **Move to the GPU only the work that is expensive and parallel, and keep the reference for everything that must match exactly.** Here that is photograph resampling. Text, overlays and transition mixing stay on the CPU because each was cheap, or because matching the reference on a GPU would have meant reproducing its quirks.

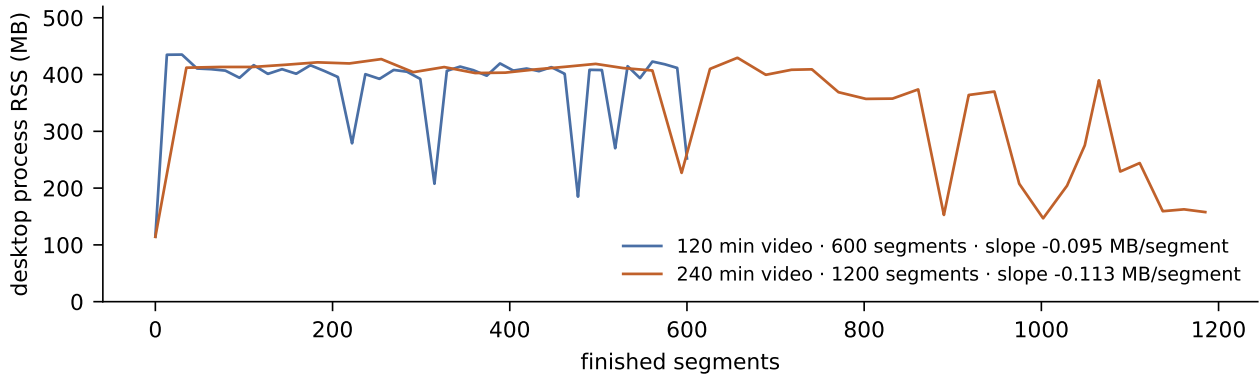


Figure 6: Resident memory of the desktop render process against finished segments. The least-squares slopes are slightly negative, so memory does not grow with segment count. A leak of a few megabytes per segment would be invisible at 13 segments and would reach a gigabyte by 1,200. The curves are the harness’s own 40-point downsample; the harness sums the desktop process and its children, so the short dips are most likely samples taken between child processes; and one final sample taken after the job’s workspace was removed is omitted.

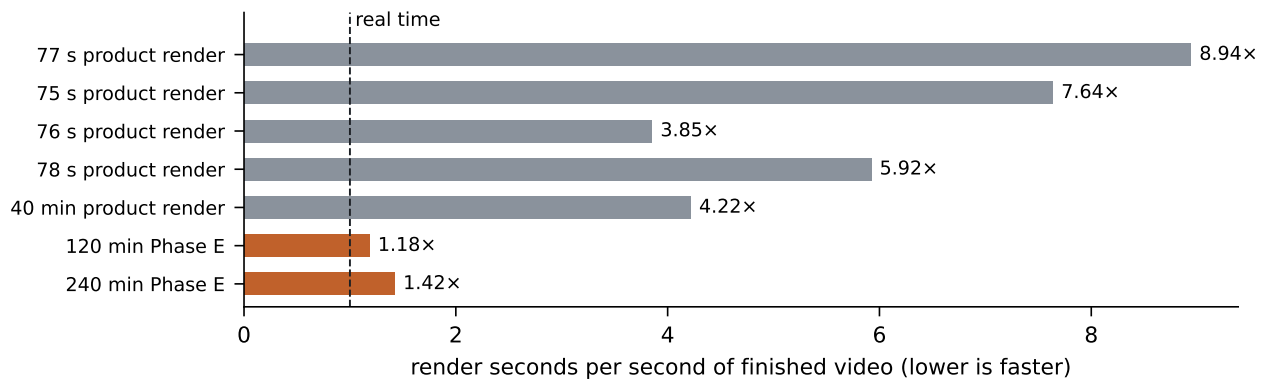


Figure 7: Render time per second of finished video. Grey: product renders from the development database (backend not recorded). Orange: long-render validation with the GPU painter. Different content and settings; not a controlled comparison.

4. **Treat correctness as a contract with a zero-outlier rule, and trust a GPU only after it has drawn a calibration frame that passes.** Most of the engineering effort in this work was spent here, and it found four real defects that unit tests on a machine without a GPU could not have found.
5. **Keep encoding on the CPU** until composition has moved and the profile has been re-measured.

11 Threats to validity and open questions

- **No controlled throughput comparison.** The repository contains a benchmark (`scripts/bench_gpu_renderer`), that times CPU and GPU composition per content type after a warm-up frame and prefers the GPU only if it is at least 10% faster. No result from it is recorded. The frames-per-second table in the project’s GPU documentation is explicitly labelled as an illustration of the output format, and it is not reproduced here.
- **One device.** All GPU evidence comes from a single GTX 1650 and driver. AMD, Intel and macOS (where OpenGL is deprecated and capped at 4.1) are untested.
- **Small calibration frames.** Equivalence was measured at 320×180. The long renders ran at 1080p but checked duration and memory, not pixels.

- **Synthetic long-render content.** The fixture uses one test-pattern photograph and typography, not real footage.
- **Profile provenance.** The 77/23 split and the 12→26 fps result are recorded in the code and documentation. The raw profiler output is not in the repository.
- **Contexts per process.** Eight segment workers create eight GL contexts. Whether fewer workers with more work each would be faster has not been measured.

12 Future work

The next steps are, first, to run the per-content benchmark at 1080p and 4K and publish the results. Second, to measure worker count against GPU-context count. Third, to move the animation-engine primitives onto the GPU without changing the `DisplayList` contract. Fourth, to re-profile after that and ask again whether hardware encoding now pays, and within what session limits. Finally, to evaluate `wgpu` (D3D12, Vulkan, Metal) as the portable successor to OpenGL.

13 Conclusion

The fastest way to render this workload was not to reach for the GPU first. Profiling showed that encoding was the smaller cost and that much of composition was avoidable work. Segmenting the render on exact frame boundaries made it resumable and parallel in one step. A narrow GPU compositor, trusted only after it reproduces the reference within two levels with no outliers, then took over the one expensive, parallel operation left. The first hardware run failed in a way the pixel counts fully explained; the second passed every scene, and the system rendered four hours of video with flat memory and exact duration. What remains is a controlled measurement of the GPU speed-up itself, and this report leaves that number blank rather than guessing it.

Acknowledgements. The design, code and documentation of Voice-to-Video were developed with the help of AI coding assistants. The measurements reported here were produced by the project’s own harnesses on the author’s hardware.

References

- [1] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.
- [2] C. E. Duchon. Lanczos filtering in one and two dimensions. *Journal of Applied Meteorology*, 18(8):1016–1022, 1979.
- [3] K. Turkowski. Filters for common resampling tasks. In A. S. Glassner, editor, *Graphics Gems*, pages 147–165. Academic Press, 1990.
- [4] T. Porter and T. Duff. Compositing digital images. *Computer Graphics (SIGGRAPH ’84)*, 18(3):253–259, 1984.
- [5] D. P. Mitchell and A. N. Netravali. Reconstruction filters in computer graphics. *Computer Graphics (SIGGRAPH ’88)*, 22(4):221–228, 1988.

A Provenance of every number

Claim	Source in the repository
77% compose / 23% encode; $1.3\times$ ceiling; 12→26 fps; NVENC session limits Text 0.7% of composition 10 ms of a 42 ms frame Vignette 19 ms/frame, 43 s per 76 s video 12 s segments; ~30 ms FFmpeg start; 40-min video at 4× its length Eight-worker cap	src/vtv/adapters/media/compute.py; docs/GPU_RENDERING.md docs/GPU_RENDERING.md; gpuPainter.py docstring src/vtv/animation/painter.py src/vtv/animation/canvas.py src/vtv/adapters/render/segments.py src/vtv/contracts/render.py (MAX_RENDER_WORKERS) docs/VERIFY_PHASE_C.md src/vtv/adapters/render/gpuPainter.py
30 s render in ~21 s on eight cores Lanczos vs bilinear up to 40 (mean 8); one pass 60 vs two passes 0; 64 taps; $10\times$ limit Tolerance 2, zero outliers, 23 scenes	 src/vtv/animation/equivalence.py; gpu_probe.py
First run 15/23; four defects; 18,432 + 2,304 = 20,736 23/23 after fixes; per-scene figures; device strings Figure 5 (earlier failing run) 60 min in 71 min, 296 segments, URL expiry Two- and four-hour runs; RSS curve	docs/GPU_RENDERING.md; gpuPainter.py docstring _to_delete/session-2026-09-05/gpu-diagnosis/report.json PNGs in the same folder; statistics recomputed docs/VERIFY_PHASE_D.md var/phase-e-report.json, var/phase-e-report.md
Long-render fixture	scripts/verify_long_render.py; scripts/seed_device_job.py
Product render durations and times Module, line and test counts	var/vtv.db, render_job documents counted from src/ and tests/

B Reproducing the GPU results

```
pip install moderngl
python -c "from vtv.adapters.render.gpu_probe import describe; print(describe())"
python -m unittest tests.test_painter_equivalence -v
python scripts/diagnose_gpu_painter.py --photo      # cpu.png / gpu.png / diff.png + report.json
python scripts/verify_gpu_render.py                # CPU-only, GPU-only, GPU with injected loss
python scripts/verify_long_render.py --minutes 120,240
python scripts/bench_gpu_renderer.py                # not yet run: the missing measurement
```